



# SHRI NARAYAN TRADERS

**Full Stack Business Management System**

---

## PROJECT PORTFOLIO & REPORT

Submitted in partial fulfillment of the requirements for the degree of

**Bachelor of Computer Applications (BCA)**

Prepared and Developed By:

**SATYA**

Academic Session: 2025 - 2026

Department of Computer Science & Applications

# BONAFIDE CERTIFICATE

This is to certify that the project work titled "Shri Narayan Traders Business Management System" is a bonafide record of full-stack engineering work carried out by Satya in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications (BCA) for the academic session 2025-2026.

This project represents original work and demonstrates practical application of modern web architectures, API designs, databases, and secure authentication methods under the guidelines of the college faculty.

**STUDENT SIGNATURE**

Satya, BCA Candidate

**PROJECT GUIDE**

Internal Supervisor

**HEAD OF  
DEPARTMENT**

HOD Computer Applications

# DECLARATION

---

I, Satya, student of Bachelor of Computer Applications (BCA) for the academic session 2025-2026, hereby declare that the project portfolio report titled "Shri Narayan Traders Business Management System" is a result of my own original work.

All tools, open-source libraries, database servers, web frameworks, and cloud service endpoints referenced in this work have been properly cited and credited. This report has not been submitted previously to any other institution for the award of any degree or diploma.

Date: June 23, 2026

Place: Munger, Bihar

---

**SATYA**

BCA Roll No. 25BCA08

# ACKNOWLEDGEMENT

---

I express my profound gratitude to the college faculty and internal supervisor for providing valuable mentorship, constant encouragement, and technical feedback throughout the development of the "Shri Narayan Traders Business Management System".

I am deeply indebted to the open-source community for creating powerful tools and robust runtimes such as Node.js, Express.js, and PDFKit, which formed the foundational stack of this application.

Special thanks to Google for providing access to the Gemini AI API, enabling dynamic context-aware description generations and smart email writing. I also acknowledge the MongoDB Atlas and Cloudinary support teams for providing high-reliability cloud database and image media storage services that facilitated successful project deployments.

Lastly, I thank my family and peers for their constant support and understanding during the design and coding stages of this portfolio system.

---

**SATYA**

# TABLE OF CONTENTS

|            |                              |    |
|------------|------------------------------|----|
| CHAPTER 1  | PROJECT OVERVIEW .....       | 6  |
| CHAPTER 2  | TECHNOLOGY STACK .....       | 8  |
| CHAPTER 3  | SYSTEM ARCHITECTURE .....    | 10 |
| CHAPTER 4  | DATABASE DESIGN .....        | 13 |
| CHAPTER 5  | API DOCUMENTATION .....      | 16 |
| CHAPTER 6  | FEATURES EXPLAINED .....     | 19 |
| CHAPTER 7  | USER MANUAL .....            | 23 |
| CHAPTER 8  | SYSTEM TESTING .....         | 26 |
| CHAPTER 9  | CHALLENGES & SOLUTIONS ..... | 28 |
| CHAPTER 10 | FUTURE SCOPE .....           | 29 |
| CHAPTER 11 | CONCLUSION .....             | 30 |
| CHAPTER 12 | BIBLIOGRAPHY .....           | 31 |

## CHAPTER 1

# PROJECT OVERVIEW

---

### 1.1 Introduction

Shri Narayan Traders is a registered hardware supplier, furniture manufacturer, and aluminium fabrication works business located in Kauda madan, Dilawer Pur, Munger, Bihar (PIN 811201). Established in 1998, the business has grown significantly, serving residential, commercial, and retail clients across the region.

To digitize the legacy systems, this full-stack business management application was developed. The system serves as an enterprise-grade digital solution to handle invoice creations, real-time catalog checking, inventory alerts, bulk dealer portals, and general enquiries logging. It enables the store to offer a streamlined, automated, and secure buying workflow.

### 1.2 Problem Statement

Before the installation of this digital portal, Shri Narayan Traders operated through traditional manual workflows, leading to major operational challenges:

1. **Manual Billing:** Generating handwritten cash memos was slow and vulnerable to calculations errors. Hand-computing SGST/CGST tax splits for high-volume invoices caused administrative overhead.
2. **Lack of Stock Visibility:** The business had no digital ledger to track inventory counts. This caused frequent stock-outs or over-ordering of slow-moving goods.
3. **Inquiries Management:** Customer catalog requests were handled verbally or via phone, resulting in lost inquiries and long delays.
4. **Lack of Dealer Interface:** Wholesale dealers had to make phone calls to negotiate pricing, leading to discrepancies and communication errors.

## 1.3 Proposed Solution

The proposed full-stack application digitizes all administrative and customer-facing interactions. Key features of the system include:

1. Automated GST Invoicing (POS): An online terminal enables sales staff to build print-ready PDF invoices. CGST, SGST, discounts, and total amounts are calculated dynamically.
2. Real-Time Inventory Control: The system decrements stock counts automatically during checkout. Low stock alert emails are triggered instantly to prevent inventory exhaustion.
3. Multi-role Admin Dashboard: The owner can monitor sales statistics, review logs, approve reviews, adjust coordinate settings, and register staff accounts from a single unified portal.
4. Dealer Portal: Registered wholesalers can log in to view customized dealer-rate catalogs and directly submit bulk orders online.

## 1.4 Project Scope

The scope of the Shri Narayan Traders Business Management System includes:

- Customer Front: Public website presenting dynamic product catalogs, filterable galleries, custom WhatsApp inquiries, customer reviews, and order tracking services.
- Admin Console: Protected dashboard for product CRUD operations, GST billing, dealer registrations approval, and site-wide coordinate updates.
- Dealer Module: Secure portal for wholesale dealer registrations, status checks, and bulk checkouts.
- API & AI Integrations: Dynamic product description generation using Gemini Vision API, and automated status emails using Gemini AI and Gmail SMTP.

CHAPTER 2

# TECHNOLOGY STACK

The technology stack of the Shri Narayan Traders Business Management System is designed to offer high efficiency, modern aesthetics, reliable datastores, and robust security parameters. The application utilizes Node.js on the backend combined with a responsive HTML5/CSS3/Bootstrap 5 frontend, connected to cloud servers for optimal scalability.

## 2.1 Frontend Specifications

The frontend is built using standard technologies to ensure fast page load speeds and seamless rendering across all screen dimensions. Details are listed below:

| Technology | Version | Purpose & Application                                        |
|------------|---------|--------------------------------------------------------------|
| HTML5      | 5.0     | Page structure, semantic DOM architecture, SEO structures.   |
| CSS3       | 3.0     | Custom Navy/Gold theme styling, glassmorphism UI components. |
| Bootstrap  | 5.3.2   | Responsive grid framework, offcanvas menus, modals layout.   |
| JavaScript | ES6+    | Dynamic AJAX API fetch requests, UI interactive actions.     |
| EJS        | 3.1.9   | Server-side templating engine to inject database variables.  |
| Chart.js   | 4.4.1   | Renders real-time statistics graphs on the Admin panel.      |

## 2.2 Backend & Database Stack

The server-side runs on Node.js using Express.js as the core API framework. Authentic security filters and dual-mode database layers are configured:

| Technology      | Version | Description & Integration                                      |
|-----------------|---------|----------------------------------------------------------------|
| Node.js         | v20.x+  | Fast, asynchronous, single-threaded V8 runtime environment.    |
| Express.js      | v5.2.x  | Web application framework handling REST routing & middlewares. |
| JSON Web Token  | v9.0.2  | Secure stateless tokens stored in HTTP-only cookies.           |
| bcryptjs        | v2.4.3  | Password salting and hashing algorithm (10 rounds).            |
| MongoDB Atlas   | v7.3.0  | Production-grade cloud database hosting user data.             |
| NeDB (Promises) | v6.2.3  | Local datastore fallback file system for offline running.      |
| PDFKit          | v0.19.1 | Vector-based PDF generator utilized for GST invoice prints.    |

## 2.3 API & Cloud Services

To expand features, third-party APIs and storage channels are mapped:

| Service / API    | Provider         | Purpose                                                  |
|------------------|------------------|----------------------------------------------------------|
| Gemini 3.5 Flash | Google AI Studio | Generates product descriptions and custom email content. |
| Cloudinary       | Cloudinary Inc.  | Provides secure CDN hosting for catalog product images.  |
| Nodemailer       | SMTP Transport   | Delivers transactional emails using Gmail App Passwords. |
| Google Translate | Google Cloud     | Allows 9-language translation of customer interfaces.    |

## CHAPTER 3

# SYSTEM ARCHITECTURE

### 3.1 Architectural Diagram

The application follows a standard Client-Server-Database pattern. The diagram below illustrates request processing pipelines:

```
[ CLIENT (Web Browser / PWA) ]
    %
    %%% (HTTP Request: JSON/GET/POST)
[ EXPRESS.JS ROUTER & MIDDLEWARES ]
  (Rate Limiter, Parser, JWT Auth)
    %
    %%%
    [ CONTROLLERS ]
    (Business Logic & AI Engines)
      %
      %%%
      [ DATABASE ADAPTER ]
      (Automatic Fallback Switch)
        %q          %r
        %%%          %%%
[ MongoDB Cloud ]   [ Local NeDB files ]
(Atlas Production)  (Offline Fail-safe)
```

### 3.2 Model-View-Controller (MVC) Pattern

The codebase is strictly organized using the MVC software design pattern to isolate data structures, client visual templates, and API endpoints:

- Model (M): NeDB datastore collections and MongoDB collections mapped inside "config/db.js".
- View (V): Front-end user interfaces and admin panels written in HTML5/Bootstrap 5 templates inside the "views/" folder and dynamically compiled using Embedded JavaScript (EJS).
- Controller (C): Javascript controllers inside "controllers/" that execute operations such as authorization (authController.js), billing (orderController.js), catalog modifications (productController.js), settings (settingsController.js), and AI calls (aiController.js).

### 3.3 Dual Database Adapter Architecture

A major feature of this system is the Custom Dual Database Adapter pattern. Designed for resilient operational continuity in areas with intermittent internet access (like rural Bihar), the database layer is split into production cloud and offline local storages.

When the system boots up, the database config module checks for the presence of the MONGODB\_URI environment key and triggers a connection to MongoDB Atlas. If the connection fails (e.g. timeout, DNS error, or IP binding issues), the adapter automatically handles the exception, prints a warning to logs, and routes all CRUD operations to local NeDB datastore files (users.db, products.db, orders.db, etc.) stored in the "data/" directory.

All CRUD methods (find, findOne, insert, update, remove, count) are wrapped inside a NeDB-compatible wrapper class. This guarantees that controllers interact with the database using identical syntax regardless of the active connection state.

### 3.4 JWT Token-Based Authentication Flow

The system uses stateless JSON Web Token (JWT) credentials to authenticate administrators and registered wholesale dealers:

1. Submission: The client sends a POST request containing login credentials to "/api/auth/login".
2. Verification: The server compares the password using bcrypt. If valid, it generates a JWT containing user ID, role, and username.
3. Cookie Storage: The JWT is signed with a 256-bit secret key and sent back inside an HTTP-only cookie. This blocks client-side scripts from reading the token, preventing XSS-based hijacking.
4. Request Authorization: Middlewares inspect cookies for incoming requests. If the signature checks out, user details are appended to "req.user" and processing continues; otherwise, the client is redirected to the login endpoint.

### 3.5 API Layer and JSON Standards

All transaction data and admin CRUD actions communicate via standard RESTful JSON interfaces. The base path for all system APIs is registered under `/api/`. Every route utilizes unified JSON formats to communicate with frontend AJAX scripts. This layout ensures easy integration with external mobile apps or desktop client platforms in the future.

Successful responses always return HTTP 200/201 status codes wrapped in standard format:

```
{ "success": true, "data": ... }
```

If the server captures an error, the global error-handling middleware intercepts the exception, logs details to the console, and returns JSON explaining the failure:

```
{ "success": false, "error": "Reason description" }
```

### 3.6 Security Headers & Protection Middlewares

To safeguard the web service against standard vulnerabilities, the application mounts protection filters:

- Helmet.js Content Security Policies: Mapped to set secure HTTP headers, blocking clickjacking, MIME-type sniffing, and unauthorized inline script injections.
- Global Rate Limiting: Mapped via `"express-rate-limit"` to restrict any IP to a maximum of 450 requests per 15-minute window. This limits brute-force attacks on login routes and protects the API against Denial-of-Service (DoS) flooding.

## CHAPTER 4

# DATABASE DESIGN

### 4.1 Entity Collections Relationship

The database consists of 8 main collections, managed dynamically using NeDB files locally and native MongoDB collections in production:

1. **USERS**: Stores admin, manager, sales staff, and dealer logins.
2. **PRODUCTS**: Holds the catalog item details, stock counts, and pricing specs.
3. **ORDERS**: Contains checkout item details, payment, and status coordinates.
4. **ENQUIRIES**: Stores contact form submissions from client pages.
5. **OFFERS**: Manages promotional discounts and countdown target schedules.
6. **GALLERY**: Stores files paths to showcase pictures.
7. **STATS**: Tracks visitor numbers and counts.
8. **SETTINGS**: Keeps global store variables (business email, maps, and gstin).

### 4.2 USERS Collection Schema

```
{
  "_id"      : "ObjectId / String (Primary Key)",
  "username" : "String (Unique phone number or username)",
  "password" : "String (bcrypt salted hash)",
  "role"     : "String (super-admin | admin | manager | sales | dealer)",
  "name"     : "String (Full Name)",
  "email"    : "String (Contact Address)",
  "businessName": "String (Only for dealer accounts)",
  "gstin"    : "String (Only for dealer accounts)",
  "isApproved": "Boolean (Only for dealer approvals)",
  "createdAt" : "Date"
}
```

## 4.3 PRODUCTS Collection Schema

```
{
  "_id"           : "ObjectId / String (Primary Key)",
  "name"          : "String (Product Name)",
  "category"      : "String (Hardware | Furniture | Aluminium)",
  "brand"         : "String (Manufacturer / Brand Name)",
  "description"   : "String (Rich HTML/Text Description)",
  "regularPrice"  : "Number (Retail selling price in INR)",
  "wholesalePrice": "Number (Dealer selling price in INR)",
  "originalPrice" : "Number (MRP/List price before discounts)",
  "stock"         : "Number (Available stock count)",
  "unit"          : "String (Piece | Set | Running Foot)",
  "hsnCode"       : "String (4-digit HSN code for GST classification)",
  "gstRate"       : "Number (0 | 5 | 12 | 18 | 28 percent)",
  "image"         : "String (Cloudinary secure URL or local asset path)",
  "specifications": "String (Comma-separated key-value specs)",
  "lowStockThreshold": "Number (Default 5, stock alert trigger)",
  "metaDescription": "String (SEO meta summary)",
  "featured"      : "Boolean (Highlight on index page)",
  "isNew"         : "Boolean (Display new arrival badge)",
  "isActive"      : "Boolean (Active in display lists)",
  "createdAt"     : "Date"
}
```

## 4.4 ORDERS Collection Schema

```
{
  "_id"          : "ObjectId / String (Primary Key)",
  "orderId"      : "String (Auto-generated code SNT-XXXX-YYYY)",
  "dealerId"     : "String (Link to users._id, or 'walkin')",
  "dealerName"   : "String (Customer / business name)",
  "dealerPhone"  : "String (Customer contact number)",
  "dealerGstin"  : "String (Optional customer GSTIN number)",
  "customerEmail": "String (Recipient email address)",
  "items"        : "Array [",
    {
      "productId" : "String (Link to products._id)",
      "name"       : "String (Item Name)",
      "qty"        : "Number (Quantity ordered)",
      "price"      : "Number (Rate applied)",
      "total"      : "Number (Qty * Price)",
      "gstRate"    : "Number (GST rate applied)"
    }
  ],
  "subtotal"     : "Number (Total before GST and discount)",
  "gstAmount"    : "Number (Calculated CGST + SGST tax total)",
  "discount"     : "Number (Flat discount applied)",
  "totalAmount"  : "Number (Subtotal + GST - Discount)",
  "status"       : "String (pending | confirmed | processing | shipped | delivered)",
  "paymentStatus": "String (pending | paid | partial)",
  "paymentMethod": "String (Cash | UPI | Card | Credit)",
  "shippingAddress": "String (Address coordinates)",
  "createdAt"    : "Date"
}
```

CHAPTER 5

# API DOCUMENTATION

The Shri Narayan Traders application provides a comprehensive RESTful JSON API layer. This design enables headless functionality, allowing front-end templates to query data using client-side Javascript (AJAX Fetch requests) without performing hard page refreshes.

## 5.1 Authentication REST endpoints

Handles user/admin login states, password migrations, and registration routes:

| Method | Endpoint Route            | Auth | Description                                                 |
|--------|---------------------------|------|-------------------------------------------------------------|
| POST   | /api/auth/login           | No   | Verifies user/admin credentials and registers JWT cookies   |
| POST   | /api/auth/register-dealer | No   | Registers a wholesale dealer, placing them in pending queue |
| GET    | /api/auth/logout          | Yes  | Clears authorization cookie tokens, resetting session.      |
| PUT    | /api/auth/change-password | Yes  | Updates current authenticated credentials.                  |

## 5.2 Catalog & Product APIs

Manages stock catalogs. Restricted actions require Admin/Manager authorization role:

| Method | Endpoint Route    | Role Required   | Description                                             |
|--------|-------------------|-----------------|---------------------------------------------------------|
| GET    | /api/products     | None            | List catalog products (allows category / search filter) |
| GET    | /api/products/:id | None            | Fetch details of a single product.                      |
| POST   | /api/products     | Admin / Manager | Insert a new product including photo upload             |
| PUT    | /api/products/:id | Admin / Manager | Update details, pricing or stock levels.                |
| DELETE | /api/products/:id | Admin / Manager | Remove product record from collections.                 |

## 5.3 Order & Invoicing APIs

Processes customer shopping orders, billing printouts, and status checks:

| Method | Endpoint Route          | Role Required     | Description                                      |
|--------|-------------------------|-------------------|--------------------------------------------------|
| POST   | /api/orders             | Any Login / Guest | Submit a checkout order (walkin or online)       |
| GET    | /api/orders/my          | Dealer            | Fetch order history for the active logged dealer |
| GET    | /api/orders/all         | Admin / Manager   | Retrieve complete list of orders.                |
| PUT    | /api/orders/:id/status  | Admin / Manager   | Modify order status (triggers updates email)     |
| GET    | /api/orders/:id/invoice | Admin / Staff     | Generate and stream print-ready GST PDF bill.    |

## 5.4 Settings, Enquiries & AI APIs

APIs for managing client inquiries, system settings adjustments, and AI triggers:

| Method | Endpoint Route                   | Auth                | Description                                        |
|--------|----------------------------------|---------------------|----------------------------------------------------|
| POST   | /api/enquiries                   | None                | Submit a contact form query from the site.         |
| GET    | /api/enquiries                   | Admin / Manager     | Retrieve complete inquiries records list.          |
| PATCH  | /api/enquiries/:id/status        | Admin / Manager     | Update status of enquiries (new/resolved).         |
| PUT    | /api/settings                    | Super Admin / Admin | Update coordinates, maps link, or Gemini API keys. |
| POST   | /api/ai/generate-product-details | Admin / Manager     | Analyze input or image to write description.       |
| POST   | /api/settings/test-email         | Super Admin / Admin | Trigger test email to verify SMTP connections.     |

## CHAPTER 6

# FEATURES EXPLAINED

---

### 6.1 Admin Panel Console

The administrative panel provides a centralized, secure control center for business owners and managers. Accessible only via authenticated credentials, the console adapts dynamically based on user roles:

- Super Admin & Admin: Full system access, including global settings adjustments, database backup and restoration files, staff role registration, and activity logs inspection.
- Manager: Access to products management, order processing, offer coordination, and customer reviews validation. Blocked from system back-ups and credentials updates.
- Sales Staff: Direct access to the POS GST Billing screen to search products, verify stock, and generate immediate customer invoices. Restructured menus hide backend settings.

### 6.2 Automated GST Invoicing Engine

The POS billing terminal includes an automated tax computation system:

- Tax Splits: When items are selected, the system calculates the correct GST rate (5%, 12%, 18%, or 28%), applying it to calculate the CGST (Central) and SGST (State) splits.
- Watermarked Vectors: Uses PDFKit to stream vector PDF files containing transparent background brand watermarks and QR codes directly to the client browser.
- Tax compliance: Displays store GSTIN, customer details, and invoice series format (SNT-XXXX-YYYY).

## 6.3 Gemini AI Integration Features

The application incorporates AI integrations using Google Gemini API to optimize catalog management and automate transaction communications:

1. **AI Description Generator:** When listing new inventory, the manager can submit the product name and category, optionally uploading an image. The Gemini API analyzes the product characteristics and returns structured JSON containing professional descriptions, SEO meta summaries, HSN codes, and suggested price brackets.
2. **Context-Aware Smart Emails:** The system uses Gemini to generate unique, personalized email alerts instead of rigid, hardcoded templates. For instance, when order status is changed to "Delivered", the AI generates a warm email thanking the customer, mentioning the specific items purchased, and asking them to submit a review on Google Business Profile. If status is "Cancelled", it shifts to an empathetic, apologetic tone offering customer service contacts.

## 6.4 Dual Database Adapter Layer

To maintain operational stability under poor internet environments, the custom database layer dynamically handles failovers. The wrapper intercepts MongoDB driver connection drops and channels all reads, writes, and counts to local NeDB datastore files in the "data/" folder.

This failover occurs without throwing runtime exceptions, ensuring the POS terminal and online store remain responsive. Once the cloud database connection is restored, the data is easily synchronized.

## 6.5 Security Hardening Implementation

The application adheres to OWASP security guidelines to protect data from vulnerabilities:

- Password Encryption: Cryptographic hashing of passwords is executed via bcrypt with a work factor of 10 salt rounds. Plain text credentials are never saved.
- Cookie Tokens protection: JWT tokens are stored in cookies configured with HTTP-Only, SameSite=Strict, and Secure flags. This completely mitigates Cross-Site Scripting (XSS) token theft and blocks CSRF hijacking.
- Boundary Validations: API controllers validate numeric inputs, preventing negative pricing values, negative inventory counts, or illegal quantities checkouts.
- Payload Restrictions: Limits incoming JSON and URL-encoded payloads to a maximum of 1MB, preventing buffer exhaustion and server memory DoS crashes.

## 6.6 Progressive Web App (PWA) Features

To offer an app-like experience on mobile interfaces, the frontend includes a Web App Manifest. Users can install the site as an application on Android or iOS home screens, which hides browser address headers, enables offline rendering of cached assets, and ensures faster loading times over cellular networks.

## 6.7 Dynamic Site Configuration Panel

Rather than hardcoding business coordinates, contact details, announcement bars, or payment banking coordinates inside EJS views, the entire frontend queries settings from the database dynamically:

- General Coordinates: Settings page enables updating store hours, address, phone numbers, and WhatsApp numbers in one centralized location.
- UPI QR Graphic Coordinates: The POS GST Invoice pulls the payment bank name, account number, IFSC, and UPI ID dynamically, compiling a real-time QR graphic for immediate scan pay checkouts.
- SEO metadata configurations: announcements bars and maps link targets update instantly site-wide on form save.

## 6.8 Audit Logs & Backup consoles

For business accountability and operations safety:

- Logs: Every change state (login, logout, product CRUD, settings updates, billing) is automatically written to a database log history with username and timestamp coordinates.
- Backups: One-click backup triggers compile all database tables into a single structured JSON archive file for safe local downloads. System restoration is executed by uploading the backup JSON.

## CHAPTER 7

# USER MANUAL

---

## 7.1 Admin Management Guide

Follow these steps to operate the business management backend:

1. **Secure Authentication Login:** Navigate to the secret URL path "/8340262401". Enter the registered username (the phone number "8340262401") and password. Click "Login" to access the main Dashboard.
2. **Inventory Catalog Entry:** Go to the "Products" tab on the left sidebar. Click "Add Product". Input the product name, brand, category, retail price, wholesale price, and current stock level. Select the product photo to upload.
3. **AI Catalog Optimization:** While adding a product, click "Generate with AI". The system queries Gemini, automatically pre-filling the description text area, SEO keywords, HSN codes, and tax rates based on the category.
4. **Real-time POS Billing:** Open the "GST Billing" screen. Search products in the search bar. Click "Add to Bill" and set the quantities. Enter optional discount values. Click "Print GST Invoice" to generate the print-ready PDF.

## 7.2 Dealer Wholesale Portal Guide

For registered dealers and wholesale clients:

1. **Account Registration:** Go to the "Dealer Portal" dropdown from the website navbar. Click "Register". Fill in your business name, dealer contact name, phone, email, full address, and business GSTIN registry coordinates. Click "Submit".
2. **Admin Verification:** The registration will be placed in the pending queue. The administrator will verify your business details from the Admin panel and click "Approve" to activate your dealer account.
3. **Wholesale Buying:** Once approved, log in via "/dealer/login" using your registered email. Your dashboard will load the product catalog displaying wholesale rates instead of retail prices. Add items to your cart, set checkout quantities, and click "Submit Bulk Order". You can monitor your shipment status under "My Orders".

## 7.3 Customer Interface Guide

For regular customers visiting the online presence:

1. **Navigating the Catalog:** Open the home page. Customers can view active discount offers, filter products by categories (Hardware, Furniture, Aluminium), and read approved customer testimonials.
2. **Submit Enquiry:** If a custom item is required, open the "Request Quote" page, fill in your contact details, and input your query. Click "Send Inquiry". The system sends an email confirmation to you and alerts the admin.
3. **Track Shipments:** Navigate to the "Track Order" link in the navbar. Input your unique Order ID (e.g. SNT-1234-5678) and click "Track" to view real-time shipping status updates.

## 7.4 Multi-Language Translation Guide

To support regional users and dealers, the customer website includes translation features:

- How to Translate: Open the website menu drawer on mobile or the utility header on desktop. Click the Google Translate select dropdown.
- Supported Languages: Customers can translate the entire site content instantly into 9 regional Indian languages, including Hindi, Bengali, Tamil, Telugu, Marathi, Punjabi, Gujarati, and Urdu. All static text, navigation links, and product titles translate dynamically.

## 7.5 Database Backup & Recovery Guide

For safeguarding business operations data:

- Backup: Log in as a Super Admin. Go to the "Backup & Restore" module under System Operations. Click "Download System Backup". The application compiles all collections (products, orders, users, reviews, logs) and downloads a single JSON archive file.
- Restoration: If database corruption occurs or the server is relocated, click "Choose File", select your downloaded JSON backup, and click "Upload Backup". The system parses the file and restores all records instantly.

CHAPTER 8

# SYSTEM TESTING

## 8.1 Manual Functional Test Checklist

The functional components of the full-stack system were tested manually against target behaviors. Test results are documented below:

| Feature Tested    | Test Inputs / Action            | Expected Behavior                              | Status |
|-------------------|---------------------------------|------------------------------------------------|--------|
| Admin Access      | secret URL /8340262401          | Loads login credentials form cleanly.          | PASS ' |
| Wrong Login       | Invalid password credentials    | Blocks entry, prompts red warning toast.       | PASS ' |
| AI Description    | Name: Aluminium Sliding Window  | Gemini populates description text fields.      | PASS ' |
| POS Billing       | Select items, click Add to Bill | Computes tax rates & sums grand total.         | PASS ' |
| GST Invoice       | Click Print Invoice button      | Streams formatted PDF GST invoice in browser.  | PASS ' |
| Order Tracking    | Enter SNT-2026-XXXX ID          | Displays current shipment status step tracker. | PASS ' |
| Contact Inquiries | Submit quote request form       | Sends confirmation email & admin alert.        | PASS ' |
| Low Stock alert   | Deduct stock count below limit  | Admin gets urgent email alert.                 | PASS ' |

## 8.2 Security Testing & Vulnerabilities Audit

Strict security audits were conducted to verify boundary values validation and payload injection blocks:

| Security Test Case         | Method / Payload Used                     | Test Result & Resolution                      |
|----------------------------|-------------------------------------------|-----------------------------------------------|
| SQL/NoSQL Injection        | Passed {\$ne: ""} inside input fields     | Stripped queries via parsed query sanitizers. |
| Cross-Site Scripting (XSS) | Injected <script>alert()</script> in form | HTML sanitized and tags escaped on render.    |
| Brute-Force Attack         | Triggered 100+ API calls within minutes   | Rate limiter blocks IP with HTTP 429.         |
| Unauthorized Access        | Directly navigated to /admin/settings     | Redirected to secret login /8340262401.       |
| Boundary Validation        | Submitted negative pricing values         | API returned HTTP 400 bad request error.      |

## CHAPTER 9

# CHALLENGES & SOLUTIONS

---

### Challenge 1: Termux FUSE Symlink EACCES Error

Installing npm packages on Android FUSE shared directories failed because FUSE mounts do not support symbolic links. Resolution: Mapped the "npm install --no-bin-links" option to disable bin-links creation during dependency installation.

### Challenge 2: Dual Database NeDB-MongoDB Compatibility Mismatch

MongoDB Atlas uses ObjectIds while local NeDB uses strings. Synced document IDs became complex objects inside local NeDB, leading to "Product not found" errors. Resolution: Patched the database adapter to serialize records and run a self-healing boot migration script to convert object IDs to string hexes.

### Challenge 3: Gemini JSON Parsing Exceptions

Gemini responses sometimes wrap JSON output inside markdown tick indicators (``json ... ``). Attempting to parse this threw exceptions. Resolution: Built string utility regex cleanups to strip markdown elements before running JSON.parse().

### Challenge 4: Render.com Ephemeral Disk restarts

Render free tier has temporary storage, wiping out local database files on restart. Resolution: Connected cloud MongoDB Atlas database as the primary data storage, leaving local NeDB strictly as an offline fallback.

## CHAPTER 10

# FUTURE SCOPE

---

The current version of the Shri Narayan Traders Business Management System successfully digitizes administrative and sales operations. The following modules are planned for future versions:

### 1. Customer Mobile App

Develop a native Android and iOS mobile app using React Native to allow retail customers to purchase furniture items directly online.

### 2. WhatsApp Business API

Integrate the official WhatsApp API to dispatch order confirmation PDFs and tracking updates directly to the client's phone number.

### 3. QR/Barcode Stock Scanning

Integrate barcode and QR scanners with the POS terminal to scan hardware inventory items and speed up checkout times.

### 4. Automated GST Filings

Build reports that compile tax transactions and generate GSTR-1 and GSTR-3B return templates.

### 5. Multi-Branch Coordination

Add warehouse management features to coordinate stock transfer orders across multiple shop showrooms.

### 6. Integrated Razorpay Checkout

Add online digital payment channels like UPI, cards, and wallets directly on order placement.

## CHAPTER 11

# CONCLUSION

---

This project successfully demonstrates the practical application of modern full-stack web development technologies to solve real-world operational business problems. By replacing manual paperwork with automated GST billing, real-time inventory alerts, a wholesale dealer portal, and AI-driven description generations, the "Shri Narayan Traders Business Management System" has modernized a legacy shop into a digitally-enabled enterprise.

Developing this portfolio system provided hands-on experience in:

- Designing RESTful APIs using Node.js and Express.js.
- Constructing secure authentication systems using JSON Web Tokens (JWT) and cookies.
- Structuring a custom database adapter to handle local NeDB and cloud MongoDB database failovers.
- Incorporating AI modules (Gemini API) and cloud deployment workflows (Render.com).

These practical full-stack skills are highly demanded by the modern IT industry and prepare a BCA graduate for advanced roles in software engineering and web application development.

## CHAPTER 12

# BIBLIOGRAPHY

---

The following documentation resources, guidelines, and manuals were referenced during the design and development of the system:

1. Node.js Documentation — [nodejs.org/api](https://nodejs.org/api)
2. Express.js API Guide — [expressjs.com/en/api.html](https://expressjs.com/en/api.html)
3. MongoDB Atlas Manual — [mongodb.com/docs/atlas](https://mongodb.com/docs/atlas)
4. Bootstrap 5 layout manual — [getbootstrap.com/docs/5.3](https://getbootstrap.com/docs/5.3)
5. PDFKit API reference manual — [pdfkit.org/docs/guide.pdf](https://pdfkit.org/docs/guide.pdf)
6. Google Gemini API Documentation — [ai.google.dev/docs](https://ai.google.dev/docs)
7. JSON Web Token (JWT) standards RFC 7519 — [jwt.io](https://jwt.io)
8. MDN Web Docs (JavaScript reference) — [developer.mozilla.org](https://developer.mozilla.org)
9. Cloudinary Image Management Guide — [cloudinary.com/documentation](https://cloudinary.com/documentation)
10. NPM package repository index — [npmjs.com](https://npmjs.com)